Studio Nx

Backend(FastAPI/Python) 41 — GET · PATCH /api/v1/users/me

2026-10-12

Backend(FastAPI/Python) 41 — `GET · PATCH /api/v1/users/me`

항목	내용
파일	`studionx/backend/app/api/v1/users.py`의 `read_me()`, `update_me()`
작업	P1-08 회원 정보: 내 정보 조회와 닉네임 변경 (PRD 01 FR-106)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	40 profile image file() · 42 POST · DELETE /users/me/profile-image

1. 하는 일

요청	응답
`GET /api/v1/users/me` + `Authorization: Bearer <Access>`	200과 내 정보 7칸 (36)
`PATCH /api/v1/users/me` + `{"nickname": " Route_New "}`	200과 바뀐 내 정보 (`"nickname": "Route_New"`)
`PATCH` + `{}`	200이고, 그대로입니다
`PATCH` + `{"email": "new@example.com"}`	200이고, 그대로입니다. 이메일은 바꿀 수 없으며(FR-106), 모르는 칸은 무시합니다
`PATCH` + `{"nickname": " a "}`, 21자	422 `VALIDATION_ERROR`와 `details.nickname`
`PATCH` + 다른 사람이 쓰는 닉네임	409 `NICKNAME_TAKEN`이고, 탈퇴한 지 3일이 안 된 닉네임이면 409 `REJOIN_TOO_SOON`입니다 (37)
토큰 없음, 틀린 토큰, 탈퇴 회원	401 `UNAUTHORIZED` (06 `CurrentUser`)

2. 언제 쓰나

화면 (JS)	부르는 것
userInfo.html의 `loadMe` (P1-10)	`GET`으로 닉네임, 이메일, 사진 칸을 채웁니다
userInfo.html의 닉네임 Save changes (P1-10)	`PATCH`
myPage.html (P1-17)	대시보드 API(P1-16)가 따로 있습니다. 이 API는 회원 정보 화면용입니다

함수들이 이어지는 순서 (`PATCH`)

순서	처리하는 곳	하는 일
①	06 `CurrentUser`	토큰으로 활동 회원을 찾습니다 (아니면 401)
②	`MeUpdateRequest` (37)	본문을 검사합니다. 공백을 지운 뒤 2~20자여야 합니다 (아니면 422, 03번 처리기)
③	37 `update_me`	중복과 탈퇴 3일 규칙을 확인하고 저장합니다 (아니면 409)
④	36 `to_me_response`	7칸 응답을 만듭니다

3. 코드

import (파일 맨 위)

```
from fastapi import APIRouter, Response, UploadFile, status
from fastapi.responses import FileResponse

from app.api.deps import CurrentUser, Db
from app.schemas.common import ErrorResponse
from app.schemas.user import (
    MeResponse,
    MeUpdateRequest,
    PasswordChangeRequest,
    WithdrawRequest,
)
from app.services import user_service
```

`user_service` 줄을 이번에 추가했습니다.

함수 (경로와 응답 설정은 템플릿 그대로입니다)

```
@router.get("/me", response_model=MeResponse, responses={401: {"model": ErrorResponse}})
def read_me(user: CurrentUser) -> MeResponse:
    return user_service.to_me_response(user)


@router.patch(
    "/me",
    response_model=MeResponse,
    responses={409: {"model": ErrorResponse}, 422: {"model": ErrorResponse}},
)
def update_me(data: MeUpdateRequest, user: CurrentUser, db: Db) -> MeResponse:
    return user_service.to_me_response(user_service.update_me(db, user, data))
```

문법 풀이

import 문법	뜻
`from app.services import user_service`	모듈 전체를 가져와서 `user_service.update_me`처럼 **어느 파일의 함수인지** 보이게 합니다 (라우터 함수 이름도 `update_me`라서 겹치지 않게 하기 위해서입니다)

식	뜻
`@router.get("/me", …)`, `@router.patch(…)`	이 라우터는 `/api/v1/users`에 붙어 있으므로 `GET · PATCH /api/v1/users/me`가 됩니다
`user: CurrentUser`	06번의 의존성입니다. 토큰을 검사해서 회원을 넣어 줍니다
`data: MeUpdateRequest`	JSON 본문을 이 모양으로 검사해서 넣어 줍니다. 틀리면 함수가 호출되지도 않고 422로 응답합니다
`db: Db`	요청마다 만들어지는 DB 세션입니다 (P1-03)
`response_model=MeResponse`	응답을 이 모양으로 한 번 더 걸러서 보내고, 문서(`/docs`)에도 표시합니다

4. 흐름

GET: 토큰을 확인하고 36번으로 응답합니다.

PATCH: 토큰을 확인하고, 본문을 검사한 뒤, 37번으로 바꾸고 36번으로 응답합니다.

5. 왜 이렇게

선택	이유
라우터는 한 줄이고, 실제 일은 서비스에 둠	P1-04, 05와 같은 원칙입니다. 규칙은 서비스에서 시험하고, 라우터는 연결만 합니다
모르는 칸(`email`)은 무시	pydantic의 기본 동작입니다. 이메일을 바꾸려는 요청도 오류 없이 "바뀌지 않음"으로 끝납니다. 화면은 이메일 칸을 읽기 전용으로 보여 줍니다
바뀐 정보를 응답으로 돌려줌	화면이 다시 `GET`을 보내지 않아도 됩니다
응답에 `user_id`가 있어도 괜찮음	토큰 주인 본인에게만 가기 때문입니다 (36 §2)

6. 시험 (`tests/test_user_routes.py`에 9개를 추가해서 모두 11개, 실제 앱과 MySQL로 실행)

시험	확인한 것
`GET`	200, 7칸, 값이 맞고, 사진은 `null`이며, 응답에 비밀번호 해시 문자열이 없습니다
`GET` 토큰 없음	401 `UNAUTHORIZED`
`PATCH` 새 닉네임	200 `Route_New`이고, DB에도 저장됩니다
`PATCH {}`	200이고, 그대로입니다
`PATCH` 이메일	200이고, 이메일은 그대로입니다
`PATCH " a "`, 21자	422 `VALIDATION_ERROR`이고, `details`에 `nickname`이 담깁니다
`PATCH` 다른 사람의 닉네임 (대소문자만 다름)	409 `NICKNAME_TAKEN`
`PATCH` 토큰 없음	401

전체 시험 299개가 통과했고, `ruff` 검사도 통과했습니다.